

# How to Reliably Evaluate Agent-Generated Performance Optimizations

Junming Liang · Anhui Normal University · July 2026 – present

## 1. Background and Problem

**Application.** The core programs of quantitative trading systems—market-data handling, factor computation, order and position processing—are mostly written in C++ or Rust and are highly sensitive to latency and throughput; engineers routinely iterate for gains of a few percent. They must also be strictly correct: a subtle change in logic can directly affect orders and positions, while existing tests usually cover only typical inputs, so passing them does not mean nothing is broken. Optimization here means being faster without being wrong.

**Workflow.** I carry out such optimizations with multiple coding agents: a profiler such as `perf` locates the hotspots; an agent, given the hotspot, the relevant code and the input/output requirements, proposes a candidate patch; and the patch is compiled, checked, measured, and then kept, repaired or discarded. Agents can propose many candidates in a short time, so the bottleneck becomes the decision: is a patch a genuine improvement, and is it worth accepting?

**Why this decision is hard.** Low-latency developers I talk with agree that optimization presupposes an accurate evaluation standard, and that this standard is the hardest part to build:

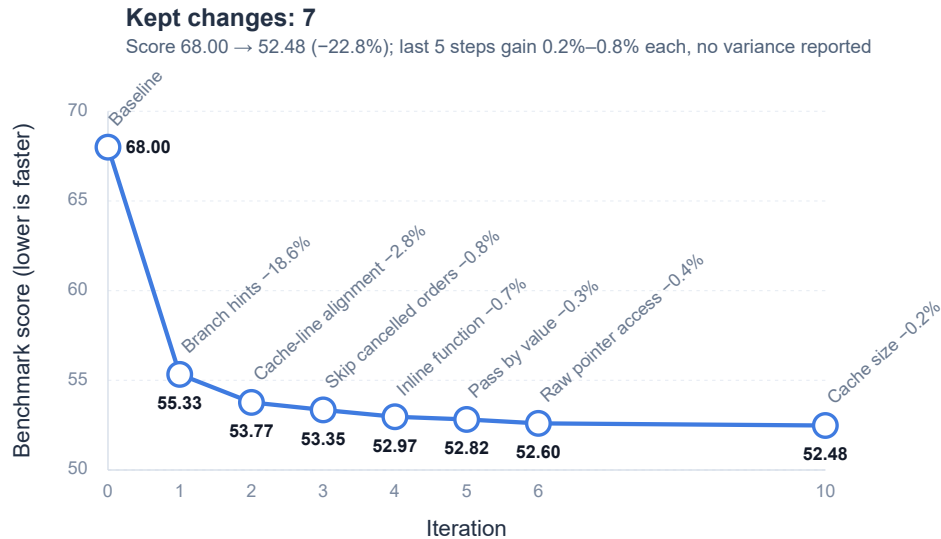
- **Measurements are noisy.** On my workload, repeated runs of the same program on the same machine vary with a standard deviation of 7–10% of the mean without CPU pinning (about 5% with pinning on the same day), while many optimizations gain only a few percent. Environment isolation—CPU pinning, process isolation, real-time scheduling—reduces the noise, and production machines for low-latency trading are typically configured this way; shared cloud and CI environments often cannot be isolated. Either way, the noise does not disappear.
- **Estimates can be biased,** wrong in one direction however many runs are made. A candidate that always runs after the baseline may benefit from warm caches; concurrent and sequential runs can give different gains; and a test environment unlike production can mislead. Selection adds more: among 20 useless patches, one or two will look faster by chance, and stopping at the first significant result pushes the error rate far above the nominal 5%.
- **The budget is limited.** Every extra minute spent on one candidate is a minute not spent on another; the more candidates, the sharper this trade-off.

**Research question.** *When the evaluation budget is limited, what evidence is sufficient to accept an agent-generated performance optimization?* The problem is not unique to my work. [auto\\_optimization](#), a public project inspired by Karpathy’s autoresearch, lets an agent optimize an order-book program, keeping any change that measures faster than the current best and passes the tests. Five of the seven changes it kept gained only 0.2–0.8% each; its benchmark script configures environment isolation, but no measurement variability is reported, so the records alone cannot confirm these small gains (Figure 1). [Chen et al. \(2026\)](#) found that many official reference patches in published agent performance benchmarks, replayed on other machines, do not consistently meet the benchmarks’ own validity criteria.

**Goal.** I treat the two risks differently. **Correctness is an absolute veto:** a candidate that fails any check is rejected or sent back for repair, and speed never buys it back. **Performance is a statistical judgment:** control how often changes without real benefit are accepted, attach an uncertainty to real gains, and treat “not confirmed within this budget” as an explicit outcome.

## 2. The Evaluation Pipeline

An implementation agent proposes and repairs patches, another agent reviews the code and evidence, and scripts run the checks and measurements and issue verdicts under frozen rules. An agent judging its own patch would be grading its own work, so rules are fixed in advance and all raw records are kept for independent recomputation; patch hashes are verified before every run, so nothing stale slips in.



**Figure 1.** The public `auto_optimization` loop. A change is kept if it beats the current best and passes 14 unit tests; in 30 minutes the agent ran 20 iterations and kept 7, discarding the rest (slower, failed builds or crashes). The benchmark script configures CPU pinning, process isolation and real-time scheduling; each point is the median of 40 runs. Score: mean time for a batch of about 36,000 order messages, in units of 10  $\mu$ s.

**Step 1: Correctness first.** On the test workload, the candidate’s final program state must match the original’s exactly. Because the workload holds only typical inputs, I also (1) write an *input contract* stating which inputs must be handled correctly and how malformed input is treated; (2) run differential tests on boundary and malformed inputs against the original program, judging inputs on which the original’s behavior is undefined by the contract; (3) run ASan and UBSan; and (4) have an agent other than the proposer review the patch and the compiler warnings. A failing patch is rejected or sent back however fast it is; a repaired patch is re-checked and re-measured as a new candidate.

**Step 2: Is the speedup real and large enough?**

- **Paired, windowed measurement.** The original and the candidate run back to back as a pair, in random order, and each pair’s time difference is one measurement. Consecutive pairs form windows in which both orders appear equally often, to balance run-order effects within each window; the example’s confirmation used 10 windows of 58 pairs, sized from the time budget and per-pair cost.
- **Window-level interval.** A 95% t-interval is computed from the window means (assumed approximately independent), because noise drifts over time and treating hundreds of pairs as independent would overstate confidence.
- **Two-level conclusion.** An interval wholly above zero shows a gain; wholly above a pre-set minimum effect (1% of the original’s time by default, adjustable in advance), the gain is worth keeping. Two cross-checks, the plain mean and the number of pairs won, must agree or be explained.

**Step 3: Prevent biased estimates.**

- **Run order** is randomized and balanced; in one batch, the two orders gave gains about 57 ms apart.
- **The measurement environment follows deployment.** Some gains depend on the environment—a change that reduces cache misses may gain differently on a dedicated core than on a shared one—and in the wrong environment more runs only measure the wrong thing more precisely. So measurement is pinned and isolated if production is, and runs under the same shared conditions if the program is deployed on cloud or CI machines.
- **Selection.** Candidates that pass exploration tend to be the lucky ones, so exploration only selects; the verdict uses fresh confirmation data alone.
- **Analysis flexibility.** The metric, the threshold and the sample size are fixed before measurement, with no changes or early stopping afterwards.

**Step 4: Decide within a budget.** Each candidate gets a fixed time—about 10 minutes of exploration and 30 of confirmation—and only those whose exploration estimate reaches the minimum effect go on to confirmation. The detection capability is estimated during planning: in the example, a change of

about 1.5–2% is estimated to have roughly an 80% chance of being detected (interval above zero); a smaller true gain will likely end as “not confirmed within this budget” rather than trigger open-ended re-measurement. Every verdict reports its machine time.

### 3. Evidence: A Real Optimization

**Outcome.** The example is a public C++ limit-order-book program that simulates an exchange’s order book and matching engine; the test workload is 600,000 order instructions (new orders, cancels, modifications, market orders, etc.). The agent’s patch replaced heavyweight C++ stream parsing and formatting with `std::from_chars` and `std::to_chars`. **The verdict was accept:** the mean time to process all instructions fell from 1.83 s to 1.28 s, **a 30.0% reduction** (550.6 ms; 95% confidence interval 527.4–573.7 ms), and the program state at the end of every timed run matched the original’s. This is the time for the whole batch, not per-order latency.

**How it was reached.**

1. **The first version was rejected.** It passed the workload check and measured faster, but the reviewing agent found three defects the workload never exercises: it dropped inputs the original accepts (e.g., leading spaces), it could read uninitialized values on malformed lines, and its output buffer relied on an unstated capacity assumption, exposed by a compiler warning.
2. **The repaired version passed** 28 differential test cases, including leading spaces, numeric boundaries and malformed input, and the sanitizers found no problem introduced by the patch.
3. **It was re-measured as a new candidate.** Exploration (192 pairs) estimated a gain of about 566 ms, above the 1% threshold; confirmation (580 fresh pairs) gave the result above, with the interval’s lower bound far above the threshold (18.3 ms). Both cross-checks agreed; the candidate was faster in 577 of 580 pairs. The two stages took about 43 minutes of machine time, including builds and checks, within the pre-set limits.

### 4. Contributions and Next Steps

**Contributions.** I designed and implemented a pipeline for evaluating agent-generated optimizations in which correctness is an absolute veto and performance must be confirmed with fresh data within a pre-declared budget. There are only three outcomes—accept, reject (or send back for repair), and not confirmed within this budget—each reported with its cost. In the real case above, checks plus timing alone would have accepted a defective patch; independent review stopped it, the input problems it found became differential tests, and a substantial runtime reduction remained after repair and fresh confirmation. The statistical methods are established; what I care about is how evidence should be gathered, and verdicts made, when agents generate candidates at scale.

**Open questions and next steps.**

- **Correctness.** Whether the checks are complete is untested. Next: fuzz the differential-test inputs, and inject known bugs to measure missed bugs and wrongly rejected valid patches.
- **False acceptance.** How often the pipeline accepts a change with no real benefit is untested. Next: count how often comment-only patches that leave the compiled result unchanged are judged faster.
- **Budget.** A 30% gain is far above the 1.5–2% this budget can resolve, so the budget went untested. Next: slow the program by a known 1–5% to find the smallest gain a given budget confirms, on pinned machines and in cloud and CI.
- **Bias.** On the repaired version, a first comparison with concurrent runs was inconclusive, and other days are untested. Next: repeat confirmation across days and with concurrent runs; when several candidates compete, confirm the winner on fresh data, with a threshold that tightens as their number grows.

Longer term, I want to study whether feeding this evidence back to the agents improves their patches.